Method overview (what problem I am solving)

I am learning a grid-agnostic, closed-loop forecaster for a 1D field $(u(x,t))$ discretized on a periodic spatial grid. Given the last few observed frames, the model predicts the next-step increment (Δu), and I roll it forward autoregressively for (K) steps. The key challenge is compounding error in closed-loop rollouts, so I train with (1) scheduled sampling, (2) multi-step supervision using "jump training", (3) spectral and phase-alignment losses, and optionally (4) a stabilizer network trained on top of a frozen forecaster.

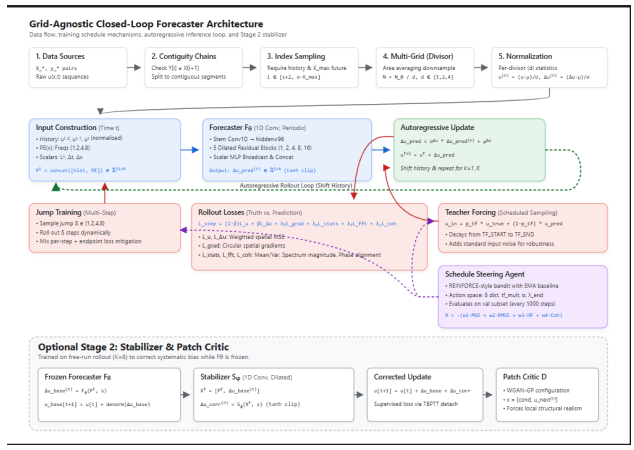

Figure 1: Architecture Diagram

Notation and discretization

Let the spatial grid have (N) cells. At time index (t), the state is

$$\mathbf{u}_t \in \mathbb{R}^N, \quad \mathbf{u}_t[i] \approx u(x_i, t).$$

The domain length is (L_x), and the grid spacing is

$$\Delta x = \frac{L_x}{N}.$$

I treat the domain as periodic, so spatial derivatives and convolutions use circular boundary conditions.

Multi-resolution / grid agnostic training

I also train on coarsened grids using "divisors" $(d \in \{1,2,4\})$. If the base grid has (N_0) points, then the coarsened grid has $(N = N_0/d)$. I downsample by area-averaging:

$$\mathrm{Down}_d(\mathbf{u})[j] = \frac{1}{d}\sum_{r=0}^{d-1} \mathbf{u}[jd + r].$$

This forces the same model to work on multiple (N) values. I also pass (Δx) as an explicit scalar so the network can condition on resolution.

Data organization and chain construction

The raw data is stored as supervised pairs $((X[i], Y[i]))$ where typically $(Y[i] \approx X[i+1])$. To do closed-loop rollouts, I need contiguous temporal sequences. I reconstruct "chains" by checking adjacency consistency:

$$\text{next_ok}[i] = \mathbf{1}\left(\frac{1}{N}\|Y[i] - X[i+1]\|_1 < \varepsilon\right),$$

with ($\varepsilon = 0.002$). If the adjacency match rate is high, I split the dataset into maximal contiguous segments (chains). Each chain ($[s, e]$) is split internally into train, val, test by fixed fractions (70%, 15%, 15%) so that each split still contains contiguous subsequences.

Sampling indices for closed-loop training

For a rollout horizon ($K_{\max}$), an index (i) is valid only if I can access (u_{i-2}, u_{i-1}, u_i) and ($K_{\max}$) future frames. So for a segment ($[s, e]$), I sample

$$i \in [s + 2,\ e - K_{\max}].$$

Model inputs (exact tensors and shapes)

Per training sample at index (i), I build:

History frames (physical units):

- ($u_{t-2} = X[i - 2]$)
- ($u_{t-1} = X[i - 1]$)
- ($u_t = X[i]$)

Future rollout targets:

- ($\{u_{t+1}, \ldots, u_{t+K_{\max}}\} = X[i + 1 : i + 1 + K_{\max}]$)

Positional encoding (Fourier features)

For grid points ($x_j = (j + 0.5)\Delta x$), I normalize to ($[0, 1]$):

$$\tilde{x}_j = \frac{x_j - x_{\min}}{x_{\max} - x_{\min}}.$$

For frequencies ($\mathcal{F} = \{1, 2, 4, 8\}$), I compute

$$\mathrm{PE}_{2k}(j) = \sin(2\pi f_k \tilde{x}_j), \quad \mathrm{PE}_{2k+1}(j) = \cos(2\pi f_k \tilde{x}_j).$$

This gives ($2|\mathcal{F}| = 8$) PE channels.

Final profile tensor

I concatenate 3 history channels plus PE:

$$\mathbf{P}_t = \mathrm{concat}\,[u_{t-2}, u_{t-1}, u_t, \mathrm{PE}] \in \mathbb{R}^{C_{in} \times N},$$

where

$$C_{in} = 3 + 2|\mathcal{F}| = 11.$$

Scalar context vector

I pass scalars:

$$\mathbf{s} = [L_x,\ \Delta t,\ \Delta x] \in \mathbb{R}^S,$$

with ($S = 1 + \mathbf{1}[\Delta t \text{ exists}] + \mathbf{1}[\text{include } \Delta x]$). In my configs, INCLUDE_DX_SCALAR = True, so typically ($S = 3$) if (Δt) exists.

Width weights for spatially weighted loss

I use ($w_j = \Delta x$) as quadrature weights so the MSE approximates an integral.

Returned dataset shapes

For each divisor (d):

- profiles_raw: ($[B, C_{in}, N]$)
- future_raw: ($[B, K_{\max}, N]$)
- scalars: ($[B, S]$)
- width: ($[B, N]$)

Normalization (per divisor)

For each divisor (d), I compute stats on train segments:

State normalization:

$$u^{(n)} = \frac{u - \mu_u^{(d)}}{\sigma_u^{(d)}}.$$

Increment normalization:

$$\Delta u = u_{t+1} - u_t, \quad \Delta u^{(n)} = \frac{\Delta u - \mu_{\Delta u}^{(d)}}{\sigma_{\Delta u}^{(d)}}.$$

I normalize the input history channels using $((\mu_u^{(d)}, \sigma_u^{(d)}))$. The network predicts normalized ($\Delta u^{(n)}$), then I de-normalize it back to physical units before updating the state.

Forecaster architecture (F_θ)

Goal: predict one-step increment (Δu).

Core autoregressive update:

$$\widehat{\Delta u}_t = \sigma_{\Delta u} F_\theta(\mathbf{P}_t^{(n)}, \mathbf{s}) + \mu_{\Delta u}, \quad \hat{u}_{t+1} = u_t + \widehat{\Delta u}_t.$$

Network structure (1D conv, periodic) Input:

- profiles: ($[B, 11, N]$)
- scalars: ($[B, S]$)

Backbone:

- stem: Conv1d(11 $\rightarrow$ hidden=96, k=3, circular padding) + norm + SiLU + dropout
- 5 residual dilated blocks with dilation (1,2,4,8,16)
- Each block: Conv1d(k=3,dilation=d,circular) + norm + SiLU + dropout, then Conv1d(k=3) + norm + dropout, residual add, SiLU.

Scalar injection:

- scalars MLP: $(S \to 128 \to 32)$
- broadcast to $([B, 32, N])$ and concatenate with conv features to get $([B, 96 + 32, N])$

Head:

- 1x1 conv + norm + SiLU + dropout
- 1x1 conv to output channel 1: $(\Delta u^{(n)} \in \mathbb{R}^{B\times 1\times N})$

Output clipping: I clip the normalized increment using a tanh bound:

$$\Delta u^{(n)} \leftarrow c \tanh\left(\frac{\Delta u^{(n)}}{c}\right),$$

with DU_CLIP_N = 5.0.

Norm choice: NORM_TYPE = "batch" (optionally "layer" supported).

Training objective (closed-loop + spectral constraints)

Weighted spatial MSE For any prediction $(\hat{u})$ and target (u):

$$\mathrm{WMSE}(\hat{u}, u) = \frac{\sum_j w_j(\hat{u}_j - u_j)^2}{\sum_j w_j}.$$

Per-step losses (physical units) At a rollout step, given (u_{in}) and predicted $(u_{\text{pred}} = u_{\text{in}} + \Delta u_{\text{pred}})$, with ground truth (u_{true}):

State loss:

$$L_u = \mathrm{WMSE}(u_{\text{pred}}, u_{\text{true}}).$$

Increment loss: True increment relative to the input is $(\Delta u_{\text{true}} = u_{\text{true}} - u_{\text{in}})$. So

$$L_{\Delta u} = \mathrm{WMSE}(\Delta u_{\text{pred}}, \Delta u_{\text{true}}).$$

Gradient matching loss (circular finite difference)

$$\nabla u[j] = u[j+1] - u[j] \quad \text{(wraparound)}, \quad L_\nabla = \mathrm{WMSE}(\nabla u_{\text{pred}}, \nabla u_{\text{true}}).$$

Mean/variance stats loss

$$L_{\text{stats}} = |\mu(u_{\text{pred}}) - \mu(u_{\text{true}})|^2 + |\sigma(u_{\text{pred}}) - \sigma(u_{\text{true}})|^2.$$

FFT magnitude loss (spectrum matching) Let $(Z = \mathrm{rFFT}(u))$ with FFT_NORM="ortho". Let $(M = |Z|)$. Then:

$$L_{\text{fft}} = \mathbb{E}_f\left[w_f(M_{\text{pred}}(f) - M_{\text{true}}(f))^2\right],$$

with a mild high-frequency upweighting $(w_f \in [0.5, 1.5])$.

Coherence (phase alignment) loss Let (Z_p, Z_t) be FFTs of prediction and truth. I compute:

$$\mathrm{coh}(f) = 1 - \Re\left(\frac{Z_p(f)\overline{Z_t(f)}}{|Z_p(f)||Z_t(f)| + \epsilon}\right), \quad L_{\text{coh}} = \mathbb{E}_f[w_f \mathrm{coh}(f)].$$

Total per-step loss I mix state and increment losses using BETA_DU = 0.50:

$$L_{\text{step}} = (1-\beta)L_u + \beta L_{\Delta u} + \lambda_{\nabla} L_{\nabla} + \lambda_{\text{stats}} L_{\text{stats}} + \lambda_{\text{fft}} L_{\text{fft}} + \lambda_{\text{coh}} L_{\text{coh}}.$$

My default weights:

- LAMBDA_GRAD = 0.05
- LAMBDA_STATS = 0.001
- LAMBDA_FFT = 0.02
- LAMBDA_COH = 0.01

Closed-loop training dynamics

Teacher forcing (scheduled sampling) During training rollouts, the next input is a continuous mix of truth and prediction:

$$u_{\text{next,in}} = p_{\text{tf}} u_{\text{true}} + (1-p_{\text{tf}}) u_{\text{pred}}.$$

The base schedule decays linearly:

- TF_START = 0.90
- TF_END = 0.00
- TF_DECAY_STEPS = 60,000

I also add small Gaussian noise in normalized space for robustness:

- INPUT_NOISE_STD = 0.01

This combination is specifically to reduce compounding error by gradually forcing the model to "live with its own mistakes" while still stabilizing early training.

Jump training (multi-step supervision without exploding horizons) Rather than always rolling out exactly (K) steps, I sample a jump length (δ) from candidates ($\{1, 2, 4, 8\}$) (capped by MAX_K). Starting from the same initial state, I roll forward (δ) steps and compute:

Per-step aggregate loss:

$$L_{\text{per-step}} = \sum_{j=1}^{\delta} w_j L_{\text{step}}^{(j)}, \quad w_j \propto \exp(-\alpha_{\text{jump}}(j-1)).$$

Here (α_{jump}) is "jump_step_alpha".

Endpoint loss at step (δ), computed two ways:

- state endpoint: $\text{WMSE}((u_\delta^{\text{pred}}, u_\delta^{\text{true}}))$
- total jump increment: $\text{WMSE}(((u_\delta^{\text{pred}} - u_0), (u_\delta^{\text{true}} - u_0)))$

Then: JUMP_LOSS_MODE = "mix"

$$L_{\text{jump}} = L_{\text{per-step}} + \lambda_{\text{end}} L_{\text{endpoint}}.$$

My base end lambda is BASE_JUMP_ENDPOINT_LAMBDA = 0.25. This directly trains the model to be correct at intermediate steps and at the far endpoint, which is a strong mitigation for drift.

Fallback (non-jump) weighting If jump training is disabled, I weight step losses across ($k = 1..K$) as

$$W_k \propto \exp(-\alpha k)$$

with BASE_ROLLOUT_ALPHA = 0.20.

Curriculum on rollout horizon (K) I use a curriculum where I start with CURR_K = 2 and gradually increase up to MAX_K = 8. I only advance (K) if:

- the validation coverage for this (K) is high (coverage_for_K ≥ 0.95)
- I have spent enough steps at this (K): MIN_STEPS_PER_K = 20,000
- validation plateaus: PLATEAU_PATIENCE = 3 evals without improvement

This ensures the model is stable at shorter horizons before being forced to do longer horizons.

Optimization details (exact configs)

Training steps and eval:

- TOTAL_STEPS = 120,000
- EVAL_EVERY = 2,000
- LOG_EVERY = 200

Batching:

- BATCH_SIZE = 64 on CUDA, else 32
- DIVISORS sampled uniformly from $(1, 2, 4)$ each step

Optimizer settings:

- weight decay = 1e-4
- grad clip = 1.0
- dropout = 0.10

Learning rate schedule (warmup + cosine decay)

- LR_START = 2e-4
- LR_MAX = 2e-3
- LR_MIN = 2e-5
- WARMUP_STEPS = 5,000

Then cosine to LR_MIN by TOTAL_STEPS.

Schedule steering (lightweight RL / bandit over training schedules)

In addition to the base schedules, I have a "schedule steering agent" that learns which training schedule works best. It is not changing model weights directly, it is choosing hyperparameters for the next interval and updating its preferences from validation reward.

It learns:

- a categorical distribution over jump deltas ($\delta \in \{1, 2, 4, 8\}$)
- discrete choices for:
  - tf_mult in {0.7, 1.0, 1.3} (multiplies teacher forcing probability)
  - rollout_alpha in {0.05, 0.10, 0.20, 0.30}
  - jump_step_alpha in {0.0, 0.10, 0.20}
  - jump_endpoint_lambda in {0.0, 0.25, 0.50}

Every STEER_EVERY = 1000 steps, it evaluates on a small val subset (STEER_VAL_BATCHES = 6) and computes metrics:

- mseK_unif
- rmse_auc (mean RMSE across steps)
- hf_err (high-frequency energy ratio mismatch)
- coherence loss

Reward is the negative weighted sum:

$$R = -\big(w_1 \text{mseK} + w_2 \text{rmse_auc} + w_3 \text{hf_err} + w_4 \text{coh}\big),$$

with

- STEER_W_MSEK = 1.0
- STEER_W_RMSE_AUC = 1.0
- STEER_W_HF = 0.5
- STEER_W_COH = 0.5

Update rule (REINFORCE style with baseline) Let ($A = R - b$) be the advantage, where (b) is an EMA baseline (STEER_BASELINE_BETA = 0.90). For a discrete choice with logits (ℓ) and probability ($p = \text{softmax}(\ell)$), chosen action index (a):

$$\ell \leftarrow \ell + \eta A \,(e_a - p)\,.$$

For the delta distribution, I also update logits using interval counts:

$$\ell_\delta \leftarrow \ell_\delta + \eta A \,(\text{counts} - np)\,.$$

This gives an automated way to discover which rollouts and TF settings reduce drift.

Inference (what runs at test time)

At inference, I use only the forecaster (F_θ) (or (F_θ) + stabilizer, if trained). The rollout is purely autoregressive with no teacher forcing:

Given starting frames (u_{t-2}, u_{t-1}, u_t):

- for $(k = 0..T-1)$:
  - normalize history ($\rightarrow (u_{t-2}^{(n)}, u_{t-1}^{(n)}, u_t^{(n)})$)
  - form ($\mathbf{P}_t^{(n)} = [u_{t-2}^{(n)}, u_{t-1}^{(n)}, u_t^{(n)}, \mathrm{PE}]$)
  - predict (Δu_t)
  - update ($u_{t+1} = u_t + \Delta u_t$)
  - shift history and repeat

I can do this both on the fine grid (divisor 1) and on coarsened grids (divisor 2 or 4) by rebuilding the PE and dx for that grid.

Optional Stage 2: Stabilizer (S_ϕ) trained on top of frozen (F_θ)

Motivation Even with jump training and spectral losses, a forecaster can accumulate small biases over long rollouts. The stabilizer is an extra correction module that learns to cancel systematic rollout drift, while keeping (F) frozen.

Decomposition of the update Forecaster proposes a base increment:

$$\Delta u_{\text{base}} = \sigma_{\Delta u} F_\theta(\mathbf{P}_t^{(n)}, \mathbf{s}) + \mu_{\Delta u}.$$

Stabilizer predicts a correction in normalized increment space:

$$\Delta u_{\text{corr}}^{(n)} = S_\phi(\mathbf{X}_t, \mathbf{s}), \quad \Delta u_{\text{corr}} = \sigma_{\Delta u} \Delta u_{\text{corr}}^{(n)}.$$

Total update:

$$u_{t+1} = u_t + \Delta u_{\text{base}} + \Delta u_{\text{corr}}.$$

Stabilizer input channels I build:

$$\mathbf{X}_t = [u_{t-2}^{(n)}, u_{t-1}^{(n)}, u_t^{(n)}, \Delta u_{\text{base}}^{(n)}, \mathrm{PE}]$$

if INCLUDE_DU_BASE = True (my default). This yields

$$C_{in,S} = 3 + 1 + 8 = 12.$$

Stabilizer architecture It is another 1D conv residual dilated network (hidden=96, dilations 1,2,4,8) with scalar injection (16 channels broadcast), and output ($\Delta u_{\text{corr}}^{(n)}$) clipped by tanh with DU_CORR_CLIP_N = 2.0.

Stabilizer training objective I train S by rolling out for K_ROLL = 8 steps using predicted states only (free-run). The supervised loss is the same mixed per-step loss as above, computed between stabilized prediction (u_{t+1}) and truth (u_{t+1}^{true}), averaged across the rollout.

To avoid unstable long backprop, I use TBPTT_DETACH = True, meaning each rollout step detaches the state before the next step.

Adversarial realism via WGAN-GP patch critic In addition to supervised loss, I add a small adversarial term so the stabilized state looks realistic in local patches, especially for high-frequency structure.

Critic (D) sees a concatenation of the conditioning context and the proposed next normalized state:

$$x_{\text{real}} = [x_{\text{cond}}, u^{(n)}_{\text{true}}], \quad x_{\text{fake}} = [x_{\text{cond}}, u^{(n)}_{\text{pred}}],$$

where

$$x_{\text{cond}} = [u^{(n)}_{t-2}, u^{(n)}_{t-1}, u^{(n)}_{t}, \text{PE}].$$

(D) is a strided conv net producing patch scores.

WGAN-GP losses:

Critic:

$$L_D = -(\mathbb{E}[D(x_{\text{real}})] - \mathbb{E}[D(x_{\text{fake}})]) + \lambda_{gp}\mathbb{E}\left(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1\right)^2,$$

with GP_LAMBDA = 10.

Generator (stabilizer) adversarial loss:

$$L_{\text{adv}} = -\mathbb{E}[D(x_{\text{fake}})].$$

Total stabilizer loss:

$$L_S = L_{\text{sup}} + \lambda_{\text{adv}} L_{\text{adv}},$$

with LAMBDA_ADV = 0.02 (kept small so the correction remains physically aligned with truth, not just visually plausible).

At inference time, I can run either:

- forecaster only: ($u_{t+1} = u_t + \Delta u_{\text{base}}$)
- forecaster + stabilizer: ($u_{t+1} = u_t + \Delta u_{\text{base}} + \Delta u_{\text{corr}}$)

End intuition: cat video analogy

You can think of the dataset as a long cat video, except each "frame" is not pixels but a 1D signal ($u(x)$).

- The forecaster (F) is like a next-frame predictor: it looks at the last 3 frames (cat positions and motion cues) and predicts the next frame change (Δu).
- Teacher forcing is like occasionally peeking at the real next video frame during training. Early on, I peek a lot (high TF), because otherwise predictions spiral. Later, I stop peeking so the model learns to stay stable when it has to watch its own generated frames.
- Jump training is like fast-forward training: sometimes I ask the model to go 1 frame ahead, sometimes 2, 4, or 8 frames ahead, and I supervise both the intermediate frames and the final frame. This directly targets the compounding error issue because the model is trained on the same kind of multi-step drift it will face at inference.
- FFT magnitude and coherence losses are like telling the model: "not only match the cat's overall pose, but also match the texture and fine wiggles correctly", so it does not blur out the high-frequency details or create phase-shifted artifacts.
- The stabilizer (S) is like a lightweight video editor sitting on top of the generator: it does not replace the main predictor, it only applies a small correction to remove systematic artifacts that accumulate over time.
- The patch critic is like a picky viewer who does not judge the whole video at once, but zooms into small regions and says "this patch looks fake". That pressure helps the stabilizer preserve realistic local structure.